Московский авиационный институт
(национальный исследовательский университет)

Кафедра 806 «Вычислительная математика и программирование»

01.03.02 «Прикладная математика и информатика»

Объектно-ориентированное программирование (семестр 1)

ЛАБОРАТОРНАЯ РАБОТА 2

КЛАССЫ. СОЗДАНИЕ КЛАССА «МНОГОЧЛЕН»

Вариант 14

Выполнил: Яшнов В.А.

Группа: М8О-203БВ-25

Преподаватель: Дзюба Д.В.

Москва, 2026

Содержание

1 Условие задачи **2**
1.1 Цель работы . . . 2
1.2 Вариант 14 . . . 2
1.3 Требования к реализации . . . 2

2 Описание программы **3**
2.1 Архитектура проекта . . . 3
2.2 Состав класса . . . 3
2.3 Особенности реализации . . . 3
2.4 Сборка и запуск . . . 3
2.5 Исходный код . . . 4

3 Тестирование **16**
3.1 Сборка проекта CMake . . . 16
3.2 Результаты тестирования . . . 17

4 Выводы **17**

5 Список литературы **17**

1 Условие задачи

1.1 Цель работы

Написать одну законченную консольную программу на языке C++, содержащую определение класса многочлена. Класс должен соответствовать правилам ООП, обеспечивать корректное управление динамической памятью, реализовывать необходимые методы и поддерживать базовые операции.

1.2 Вариант 14

Многочлен n-й степени с массивом коэффициентов A[0]..A[n]. Необходимо реализовать:

- Конструктор по умолчанию
- Конструктор копирования
- Конструктор перемещения
- Деструктор
- Оператор присваивания с реализацией через копию (copy-and-swap idiom)
- `void print(std::ostream& os) const` — вывод многочлена в поток
- `double evaluate(double in) const` — вычисление значения многочлена в точке
- `void evaluate_many(const double* xs, double* ys, size_t n) const` — вычисление значений для массива точек
- `void add(const Polynomial& other)` — сложение с другим многочленом (по месту)
- `void mul(const Polynomial& other)` — умножение на другой многочлен (по месту)

1.3 Требования к реализации

- Хранение коэффициентов — динамический массив `double*` по степеням (A[0] — свободный член, A[i] — коэффициент при x^i)
- При наличии старших нулевых коэффициентов они должны отбрасываться (метод `trim`)
- Корректная работа с пустым многочленом (нулевая степень или все коэффициенты нулевые)
- Глубокое копирование и перемещение с корректным владением памятью
- Отсутствие утечек памяти
- Проект CMake и не менее 10 тестов Google Test

2 Описание программы

2.1 Архитектура проекта

Программа реализована на основе класса `Polynomial`, обеспечивающего работу с многочленом через динамический массив коэффициентов `double*`. Коэффициенты хранятся по степеням: `A[0]` — свободный член, `A[i]` — коэффициент при x^i. Управление памятью осуществляется вручную через `new[]`/`delete[]`, соблюдается RAII.

2.2 Состав класса

Класс `Polynomial` содержит:

Таблица 1: Методы класса `Polynomial`

Сигнатура
`Polynomial(const double* data, int degree)`
`Polynomial(const Polynomial& other)`
`Polynomial(Polynomial&& other) noexcept`
`Polynomial& operator=(Polynomial other) noexcept`
`~Polynomial()`
`void print(std::ostream& os) const`
`double evaluate(double in) const`
`void evaluate_many(const double* xs, double* ys, size_t n) const`
`void add(const Polynomial& other)`
`void mul(const Polynomial& other)`

Вспомогательные методы: `void trim()` (удаление старших нулевых коэффициентов), `void swap(Polynomial& other) noexcept`.

2.3 Особенности реализации

- Старшие нулевые коэффициенты отбрасываются методом `trim()` после операций, что исключает хранение лишних членов.
- При всех коэффициентах, равных нулю, многочлен считается пустым (`degree_ = -1`, `data_ = nullptr`).
- Оператор присваивания реализован через copy-and-swap.
- Конструктор копирования выполняет глубокое копирование.
- Перемещающий конструктор передаёт владение памятью без копирования.

2.4 Сборка и запуск

Проект собирается out-of-source через CMake: статическая библиотека `polynomial`, исполняемый файл `main` и тестовый исполняемый файл `test_polynomial`. Google Test подключается через `FetchContent` (версия 1.14.0), тесты регистрируются в CTest командой `gtest_discover_tests`.

Листинг 1: Сборка и запуск

```
cmake -S . -B build
cmake --build build
./build/main
ctest --test-dir build --output-on-failure
```

2.5 Исходный код

Система сборки

Листинг 2: CMakeLists.txt

```
# Files
cmake_minimum_required(VERSION 3.16)
project(lab2_variant14 CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

if(MSVC)
    set(LAB_WARNINGS /W4)
else()
    set(LAB_WARNINGS -Wall -Wextra -Wpedantic)
endif()

add_library(polynomial STATIC src/polynomial.cpp)
target_include_directories(polynomial PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/src)
target_compile_options(polynomial PRIVATE ${LAB_WARNINGS})

add_executable(main src/main.cpp)
target_link_libraries(main PRIVATE polynomial)
target_compile_options(main PRIVATE ${LAB_WARNINGS})

# Tests
include(FetchContent)
set(CMP0135 NEW)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
    DOWNLOAD_EXTRACT_TIMESTAMP TRUE
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)

enable_testing()
add_executable(test_polynomial tests/test_polynomial.cpp)
target_link_libraries(test_polynomial PRIVATE polynomial GTest::gtest_main)
target_compile_options(test_polynomial PRIVATE ${LAB_WARNINGS})

include(GoogleTest)
gtest_discover_tests(test_polynomial)
```

Заголовочный файл

Листинг 3: src/polynomial.h

```h
#pragma once

#include <iostream>

class Polynomial {
    private:
        double* data_ = nullptr;
        int degree_ = -1;
        void trim();
        void swap(Polynomial& other) noexcept;
    public:
        Polynomial(const double* data, int degree);
        Polynomial(const Polynomial& other);
        Polynomial(Polynomial&& other) noexcept;
        Polynomial& operator= (Polynomial other) noexcept;
        ~Polynomial();
        void print(std::ostream& os) const;
        double evaluate(double in) const;
        void evaluate_many(const double* xs, double* ys, size_t n) const;
        void add(const Polynomial& other);
        void mul(const Polynomial& other);
};
```

Реализация класса

Листинг 4: src/polynomial.cpp

```cpp
#include <algorithm>
#include <iostream>
#include <cmath>
#include "polynomial.h"

void Polynomial::trim()
{
    while (degree_ >= 0 && data_[degree_] == 0.0)
        --degree_;

    if (degree_ < 0)
    {
        delete[] data_;
        data_ = nullptr;
    }
}

void Polynomial::swap(Polynomial &other) noexcept
{
    std::swap(data_, other.data_);
    std::swap(degree_, other.degree_);
}

Polynomial::Polynomial(const double *data, int degree)
```

```
25 {
26     if (degree < 0 || data == nullptr)
27     {
28         return;
29     }
30     degree_ = degree;
31     data_ = new double[degree_ + 1];
32     for (int i = 0; i <= degree_; ++i)
33     {
34         data_[i] = data[i];
35     }
36     trim();
37 }
38
39 Polynomial::Polynomial(const Polynomial &other) : data_(new
       double[other.degree_ + 1]), degree_(other.degree_)
40 {
41     for (int i = 0; i <= degree_; ++i)
42     {
43         data_[i] = other.data_[i];
44     }
45     trim();
46 }
47
48 Polynomial::Polynomial(Polynomial &&other) noexcept : data_(other.data_),
       degree_(other.degree_)
49 {
50     other.data_ = nullptr;
51     other.degree_ = -1;
52     trim();
53 }
54
55 Polynomial &Polynomial::operator=(Polynomial other) noexcept
56 {
57     swap(other);
58     trim();
59     return *this;
60 }
61
62 Polynomial::~Polynomial()
63 {
64     delete[] data_;
65 }
66
67 void Polynomial::print(std::ostream &os) const
68 {
69     if (data_ == nullptr || degree_ == -1)
70     {
71         os << "[empty polynomial]";
72     }
73     for (int i = degree_; i >= 0; --i)
74     {
75         if (data_[i] == 0)
```

```
76          {
77              continue;
78          }
79          if (i < degree_ && data_[i] > 0)
80          {
81              os << "+ ";
82          }
83          if (data_[i] < 0)
84          {
85              os << "- ";
86          }
87          if (data_[i] != 1 && data_[i] != -1)
88          {
89              if (i > 1)
90              {
91                  os << ((data_[i] > 0) ? data_[i] : -data_[i]) << "x^" << i << "
    ";
92              }
93              else if (i == 1)
94              {
95                  os << ((data_[i] > 0) ? data_[i] : -data_[i]) << "x" << " ";
96              }
97              else
98              {
99                  os << ((data_[i] > 0) ? data_[i] : -data_[i]);
100             }
101         }
102         else
103         {
104             if (i > 1)
105             {
106                 os << "x^" << i << " ";
107             }
108             else if (i == 1)
109             {
110                 os << "x" << " ";
111             }
112             else
113             {
114                 os << "1";
115             }
116         }
117     }
118     os << "\n";
119 }
120
121 double Polynomial::evaluate(double in) const
122 {
123     double out = 0.0;
124     for (int i = degree_; i >= 0; --i)
125     {
126         out += data_[i] * std::pow(in, i);
127     }
```

```
128     return out;
129 }
130
131 void Polynomial::evaluate_many(const double *xs, double *ys, size_t n) const
132 {
133     for (size_t i = 0; i < n; ++i)
134     {
135         ys[i] = evaluate(xs[i]);
136     }
137 }
138
139 void Polynomial::add(const Polynomial &other)
140 {
141     int max_degree = std::max(degree_, other.degree_);
142     double *temp = new double[max_degree + 1]{};
143     for (int i = max_degree; i >= 0; --i)
144     {
145         if (i <= degree_ && data_ != nullptr)
146         {
147             temp[i] += data_[i];
148         }
149         if (i <= other.degree_ && other.data_ != nullptr)
150         {
151             temp[i] += other.data_[i];
152         }
153     }
154     delete[] data_;
155     data_ = temp;
156     degree_ = max_degree;
157     trim();
158 }
159
160 void Polynomial::mul(const Polynomial &other)
161 {
162     if (data_ == nullptr || other.data_ == nullptr)
163     {
164         delete[] data_;
165         data_ = nullptr;
166         degree_ = -1;
167         return;
168     }
169     int new_degree = degree_ + other.degree_;
170     double *temp = new double[new_degree + 1]{};
171     for (int i = degree_; i >= 0; --i)
172     {
173         for (int j = other.degree_; j >= 0; --j)
174         {
175             temp[i + j] += data_[i] * other.data_[j];
176         }
177     }
178     delete[] data_;
179     data_ = temp;
180     degree_ = new_degree;
```

```
    trim();
}
```

Точка входа

Листинг 5: src/main.cpp

```cpp
#include "polynomial.h"

int main() {
double c1[] = {1.0, 2.0}; // 1 + 2x
    double c2[] = {0.0, 1.0, 1.0}; // x + x^2
    Polynomial a(c1, 1);
    Polynomial b(c2, 2);
    Polynomial s(a);
    s.add(b); // 1 + 3x + x^2
    s.print(std::cout);
    std::cout << s.evaluate(2.0) << std::endl; // 1 + 6 + 4 = 11
    Polynomial m(a);
    m.mul(b); // (1 + 2x)(x + x^2) = x + 3x^2 + 2x^3
    m.print(std::cout);
    return 0;
}
```

Тесты

Листинг 6: tests/test_polynomial.cpp

```cpp
#include <gtest/gtest.h>

#include <cmath>
#include <sstream>
#include <utility>

#include "polynomial.h"

namespace
{
    std::string print_str(const Polynomial &p)
    {
        std::ostringstream os;
        p.print(os);
        return os.str();
    }
}

TEST(Polynomial, ConstructAndEvaluate)
{
    double c[] = {1.0, 2.0}; // 1 + 2x
    Polynomial p(c, 1);

    EXPECT_DOUBLE_EQ(p.evaluate(0.0), 1.0);
    EXPECT_DOUBLE_EQ(p.evaluate(2.0), 5.0);
    EXPECT_DOUBLE_EQ(p.evaluate(-1.0), -1.0);
    EXPECT_EQ(print_str(p), "2x + 1\n");
}
```

```
29
30 TEST(Polynomial, ConstructorTrimsLeadingZeros)
31 {
32     double c[] = {3.0, 2.0, 0.0, 0.0}; // 3 + 2x
33     Polynomial p(c, 3);
34
35     EXPECT_DOUBLE_EQ(p.evaluate(1.0), 5.0);
36     EXPECT_DOUBLE_EQ(p.evaluate(2.0), 7.0);
37     EXPECT_EQ(print_str(p), "2x + 3\n");
38 }
39
40 TEST(Polynomial, AllZeroCoefficientsIsEmpty)
41 {
42     double c[] = {0.0, 0.0, 0.0};
43     Polynomial p(c, 2);
44
45     EXPECT_DOUBLE_EQ(p.evaluate(10.0), 0.0);
46     EXPECT_EQ(print_str(p), "[empty polynomial]\n");
47 }
48
49 TEST(Polynomial, NullDataIsEmpty)
50 {
51     Polynomial p(nullptr, 5);
52
53     EXPECT_DOUBLE_EQ(p.evaluate(1.0), 0.0);
54     EXPECT_EQ(print_str(p), "[empty polynomial]\n");
55 }
56
57 TEST(Polynomial, EmptyPolynomialEvaluatesToZero)
58 {
59     double c[] = {0.0};
60     Polynomial p(c, 0);
61
62     EXPECT_DOUBLE_EQ(p.evaluate(0.0), 0.0);
63     EXPECT_DOUBLE_EQ(p.evaluate(-7.5), 0.0);
64     EXPECT_EQ(print_str(p), "[empty polynomial]\n");
65 }
66
67 TEST(Polynomial, CopyIsDeep)
68 {
69     double c[] = {1.0, 2.0};
70     Polynomial a(c, 1);
71     Polynomial b(a);
72
73     double ten[] = {10.0};
74     a.add(Polynomial(ten, 0));
75
76     EXPECT_DOUBLE_EQ(a.evaluate(0.0), 11.0);
77     EXPECT_DOUBLE_EQ(b.evaluate(0.0), 1.0);
78     EXPECT_EQ(print_str(b), "2x + 1\n");
79 }
80
81 TEST(Polynomial, MoveEmptiesSource)
```

```
{
    double c[] = {1.0, 2.0};
    Polynomial src(c, 1);
    Polynomial dst(std::move(src));

    EXPECT_DOUBLE_EQ(dst.evaluate(2.0), 5.0);
    EXPECT_EQ(print_str(dst), "2x + 1\n");

    EXPECT_DOUBLE_EQ(src.evaluate(100.0), 0.0);
    EXPECT_EQ(print_str(src), "[empty polynomial]\n");
}

TEST(Polynomial, CopyAssignment)
{
    double c1[] = {1.0, 2.0};
    double c2[] = {0.0, 0.0, 1.0}; // x^2
    Polynomial a(c1, 1);
    Polynomial b(c2, 2);

    a = b;
    EXPECT_EQ(print_str(a), "x^2 \n");
    EXPECT_DOUBLE_EQ(a.evaluate(3.0), 9.0);

    double five[] = {5.0};
    b.add(Polynomial(five, 0));
    EXPECT_DOUBLE_EQ(a.evaluate(2.0), 4.0);
}

TEST(Polynomial, SelfAssignmentIsSafe)
{
    double c[] = {1.0, 2.0};
    Polynomial p(c, 1);

    p = p;
    EXPECT_DOUBLE_EQ(p.evaluate(2.0), 5.0);
    EXPECT_EQ(print_str(p), "2x + 1\n");
}

TEST(Polynomial, SelfMoveAssignmentIsSafe)
{
    double c[] = {1.0, 2.0};
    Polynomial p(c, 1);
    p = std::move(p);
    EXPECT_DOUBLE_EQ(p.evaluate(2.0), 5.0);
    EXPECT_EQ(print_str(p), "2x + 1\n");
}

TEST(Polynomial, AddBasic)
{
    double c1[] = {1.0, 2.0};      // 1 + 2x
    double c2[] = {0.0, 1.0, 1.0}; // x + x^2
    Polynomial a(c1, 1);
    Polynomial b(c2, 2);
```

```
135
136     Polynomial s(a);
137     s.add(b); // 1 + 3x + x^2
138
139     EXPECT_EQ(print_str(s), "x^2 + 3x + 1\n");
140     EXPECT_DOUBLE_EQ(s.evaluate(2.0), 11.0);
141 }
142
143 TEST(Polynomial, AddWithEmptyKeepsOperands)
144 {
145     double c[] = {1.0, 2.0};
146     double z[] = {0.0};
147     Polynomial p(c, 1);
148     Polynomial empty(z, 0);
149
150     Polynomial a(p);
151     a.add(empty);
152     EXPECT_EQ(print_str(a), "2x + 1\n");
153     EXPECT_DOUBLE_EQ(a.evaluate(2.0), 5.0);
154
155     Polynomial b(empty);
156     b.add(p);
157     EXPECT_EQ(print_str(b), "2x + 1\n");
158     EXPECT_DOUBLE_EQ(b.evaluate(2.0), 5.0);
159
160     Polynomial e1(empty);
161     Polynomial e2(empty);
162     e1.add(e2);
163     EXPECT_EQ(print_str(e1), "[empty polynomial]\n");
164 }
165
166 TEST(Polynomial, AddCancellationLowersDegree)
167 {
168     double c1[] = {1.0, 1.0};  // 1 + x
169     double c2[] = {0.0, -1.0}; // -x
170     Polynomial a(c1, 1);
171     Polynomial b(c2, 1);
172
173     a.add(b); // 1
174     EXPECT_EQ(print_str(a), "1\n");
175     EXPECT_DOUBLE_EQ(a.evaluate(7.0), 1.0);
176 }
177
178 TEST(Polynomial, AddCancelsToEmpty)
179 {
180     double c1[] = {1.0, 1.0};   // 1 + x
181     double c2[] = {-1.0, -1.0}; // -1 - x
182     Polynomial a(c1, 1);
183     Polynomial b(c2, 1);
184
185     a.add(b);
186     EXPECT_EQ(print_str(a), "[empty polynomial]\n");
187     EXPECT_DOUBLE_EQ(a.evaluate(3.0), 0.0);
```

```
188 }
189
190 TEST(Polynomial, MulBasic)
191 {
192     double c1[] = {1.0, 2.0};      // 1 + 2x
193     double c2[] = {0.0, 1.0, 1.0}; // x + x^2
194     Polynomial a(c1, 1);
195     Polynomial b(c2, 2);
196
197     Polynomial m(a);
198     m.mul(b); // x + 3x^2 + 2x^3
199
200     EXPECT_EQ(print_str(m), "2x^3 + 3x^2 + x \n");
201     EXPECT_DOUBLE_EQ(m.evaluate(2.0), 30.0); // 2 + 12 + 16
202     EXPECT_DOUBLE_EQ(m.evaluate(0.0), 0.0);
203 }
204
205 TEST(Polynomial, MulByConstant)
206 {
207     double c1[] = {1.0, 2.0};
208     double c2[] = {3.0};
209     Polynomial a(c1, 1);
210     Polynomial b(c2, 0);
211
212     a.mul(b); // 3 + 6x
213     EXPECT_EQ(print_str(a), "6x + 3\n");
214     EXPECT_DOUBLE_EQ(a.evaluate(1.0), 9.0);
215 }
216
217 TEST(Polynomial, MulByEmptyGivesEmpty)
218 {
219     double c[] = {1.0, 2.0};
220     double z[] = {0.0};
221     Polynomial p(c, 1);
222     Polynomial empty(z, 0);
223
224     Polynomial a(p);
225     a.mul(empty);
226     EXPECT_EQ(print_str(a), "[empty polynomial]\n");
227
228     Polynomial b(empty);
229     b.mul(p);
230     EXPECT_EQ(print_str(b), "[empty polynomial]\n");
231 }
232
233 TEST(Polynomial, MulTwoCopiesOfEmpty)
234 {
235     double z[] = {0.0};
236     Polynomial empty(z, 0);
237     Polynomial b(empty);
238     Polynomial c(empty);
239
240     EXPECT_NO_THROW(b.mul(c));
```

```
    EXPECT_EQ(print_str(b), "[empty polynomial]\n");
    EXPECT_DOUBLE_EQ(b.evaluate(1.0), 0.0);
}

TEST(Polynomial, PrintConstantOne)
{
    double c[] = {1.0};
    Polynomial p(c, 0);
    EXPECT_EQ(print_str(p), "1\n");
}

TEST(Polynomial, PrintNegativeLeadingCoefficient)
{
    double c[] = {-2.0, 0.0, 1.0}; // -2 + x^2
    Polynomial p(c, 2);

    EXPECT_EQ(print_str(p), "x^2 - 2\n");
    EXPECT_DOUBLE_EQ(p.evaluate(0.0), -2.0);
    EXPECT_DOUBLE_EQ(p.evaluate(2.0), 2.0);
}

TEST(Polynomial, PrintUnitCoefficients)
{
    double c[] = {1.0, -1.0}; // 1 - x
    Polynomial p(c, 1);

    EXPECT_EQ(print_str(p), "- x + 1\n");
    EXPECT_DOUBLE_EQ(p.evaluate(1.0), 0.0);
}

TEST(EvaluateMany, ExampleFromVariant)
{
    double c[] = {0.0, 0.0, 1.0}; // x^2
    Polynomial p(c, 2);

    double xs[] = {-1.0, 0.0, 1.0, 2.0};
    double ys[4] = {};
    p.evaluate_many(xs, ys, 4);

    EXPECT_DOUBLE_EQ(ys[0], 1.0);
    EXPECT_DOUBLE_EQ(ys[1], 0.0);
    EXPECT_DOUBLE_EQ(ys[2], 1.0);
    EXPECT_DOUBLE_EQ(ys[3], 4.0);
}

TEST(EvaluateMany, MatchesEvaluate)
{
    double c[] = {1.0, -2.0, 0.5, 3.0};
    Polynomial p(c, 3);

    double xs[] = {-2.0, -0.5, 0.0, 0.25, 1.0, 3.7};
    double ys[6] = {};
    p.evaluate_many(xs, ys, 6);
```

```
294
295     for (size_t i = 0; i < 6; ++i)
296     {
297         EXPECT_NEAR(ys[i], p.evaluate(xs[i]), 1e-9) << "i = " << i;
298     }
299 }
300
301 TEST(EvaluateMany, ZeroCountLeavesBufferUntouched)
302 {
303     double c[] = {1.0, 2.0};
304     Polynomial p(c, 1);
305
306     double xs[] = {1.0, 2.0, 3.0};
307     double ys[] = {42.0, 42.0, 42.0};
308     p.evaluate_many(xs, ys, 0);
309
310     EXPECT_DOUBLE_EQ(ys[0], 42.0);
311     EXPECT_DOUBLE_EQ(ys[1], 42.0);
312     EXPECT_DOUBLE_EQ(ys[2], 42.0);
313 }
314
315 TEST(EvaluateMany, EmptyPolynomialGivesZeros)
316 {
317     double z[] = {0.0};
318     Polynomial p(z, 0);
319
320     double xs[] = {-5.0, 0.0, 5.0};
321     double ys[3] = {1.0, 1.0, 1.0};
322     p.evaluate_many(xs, ys, 3);
323
324     EXPECT_DOUBLE_EQ(ys[0], 0.0);
325     EXPECT_DOUBLE_EQ(ys[1], 0.0);
326     EXPECT_DOUBLE_EQ(ys[2], 0.0);
327 }
328
329 TEST(EvaluateMany, SinglePoint)
330 {
331     double c[] = {1.0, 2.0};
332     Polynomial p(c, 1);
333
334     double xs[] = {2.0};
335     double ys[] = {0.0};
336     p.evaluate_many(xs, ys, 1);
337
338     EXPECT_DOUBLE_EQ(ys[0], 5.0);
339 }
340
341 TEST(EvaluateMany, FractionalAndNegativePoints)
342 {
343     double c[] = {1.0, 2.0}; // 1 + 2x
344     Polynomial p(c, 1);
345
346     double xs[] = {-0.5, 0.25, 1.5};
```

```
347     double ys[3] = {};
348     p.evaluate_many(xs, ys, 3);
349 
350     EXPECT_DOUBLE_EQ(ys[0], 0.0);
351     EXPECT_DOUBLE_EQ(ys[1], 1.5);
352     EXPECT_DOUBLE_EQ(ys[2], 4.0);
353 }
```

3 Тестирование

3.1 Сборка проекта CMake

Файл `CMakeLists.txt` настраивает сборку статической библиотеки `polynomial`, исполняемого файла `main` и тестового исполняемого файла `test_polynomial`. Google Test подтягивается автоматически через `FetchContent`.

```
1  # Files
2  cmake_minimum_required(VERSION 3.16)
3  project(lab2_variant14 CXX)
4  
5  set(CMAKE_CXX_STANDARD 20)
6  set(CMAKE_CXX_STANDARD_REQUIRED ON)
7  set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8  
9  if(MSVC)
10     set(LAB_WARNINGS /W4)
11 else()
12     set(LAB_WARNINGS -Wall -Wextra -Wpedantic)
13 endif()
14 
15 add_library(polynomial STATIC src/polynomial.cpp)
16 target_include_directories(polynomial PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/src)
17 target_compile_options(polynomial PRIVATE ${LAB_WARNINGS})
18 
19 add_executable(main src/main.cpp)
20 target_link_libraries(main PRIVATE polynomial)
21 target_compile_options(main PRIVATE ${LAB_WARNINGS})
22 
23 # Tests
24 include(FetchContent)
25 set(CMP0135 NEW)
26 FetchContent_Declare(
27     googletest
28     URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
29     DOWNLOAD_EXTRACT_TIMESTAMP TRUE
30 )
31 set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
32 FetchContent_MakeAvailable(googletest)
33 
34 enable_testing()
35 add_executable(test_polynomial tests/test_polynomial.cpp)
36 target_link_libraries(test_polynomial PRIVATE polynomial GTest::gtest_main)
37 target_compile_options(test_polynomial PRIVATE ${LAB_WARNINGS})
38 
```

```
include(GoogleTest)
gtest_discover_tests(test_polynomial)
```

3.2 Результаты тестирования

Тесты размещены в `tests/test_polynomial.cpp` (27 тестов). Примеры проверяемых сценариев: конструирование из массива, обрезка старших нулевых (`trim`), обработка пустого/нулевого многочлена, глубокое копирование, перемещение, присваивание, печать, вычисление в точке, вычисление для массива точек, сложение и умножение многочленов.

Сборка и запуск тестов (из корня `lab_2`):

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

Все тесты успешно проходят.

4 Выводы

В ходе выполнения лабораторной работы №2 реализован класс `Polynomial` для работы с многочленами с хранением коэффициентов в динамическом массиве. Применены RAII, глубокое копирование, перемещение, идиома copy-and-swap. Реализованы методы `print`, `evaluate`, `evaluate_many`, `add`, `mul`. Корректно обрабатываются случаи пустого многочлена и старших нулевых коэффициентов (`trim`). Проект оформлен через CMake, покрыт тестами Google Test (27 тестов). Достигнута требуемая функциональность без использования запрещённых средств.

5 Список литературы

1. ISO/IEC 14882:2020 (C++20). International Organization for Standardization, 2020.
2. Страуструп Б. Язык программирования C++. Спец. изд. — М.: Бином, 2011.
3. Кормен Т. Х. и др. Алгоритмы: построение и анализ. — М.: Вильямс, 2006.
4. CMake Documentation. [Электронный ресурс]. URL: https://cmake.org/documentation/.
5. GoogleTest Documentation. [Электронный ресурс]. URL: https://google.github.io/googletest/.